

Trees Problems and Solutions:

1. Invert Binary Tree

Approach 1: Recursive

Swap left and right child recursively for all nodes.

Pseudocode:

```
TreeNode* invertTree(TreeNode* root) {
```

```
    if (!root) return nullptr;
```

```
    TreeNode* temp = root->left;
```

```
    root->left = root->right;
```

```
    root->right = temp;
```

```
    invertTree(root->left);
```

```
    invertTree(root->right);
```

```
    return root;
```

```
}
```

Dry Run:

Input: [4, 2, 7, 1, 3, 6, 9]

Swap 2 ↔ 7, then recursively swap their children.

Output: [4, 7, 2, 9, 6, 3, 1]

- Time Complexity:

$O(N)$.

- Space Complexity: $O(H)$, where H is the height of the tree (recursive stack).

2. Maximum Depth of Binary Tree

Approach 1: Recursive Depth-First Search

- Compute the maximum depth of left and right subtrees and add 1.

Pseudocode:

```
int maxDepth(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(maxDepth(root->left), maxDepth(root->right));  
}
```

Dry Run:

Input: [3, 9, 20, null, null, 15, 7]

Depth of left = 1, right = 2 + 1 = 3.

Output: 3

- Time Complexity: $O(N)$.
- Space Complexity: $O(H)$.

3. Diameter of Binary

Tree

Approach 1: Recursive DFS

- For each node, compute the diameter as the sum of the left and right subtree heights.

Pseudocode:

```
int diameter(TreeNode* root, int &maxDiameter) {  
    if (!root) return 0;  
    int left = diameter(root->left, maxDiameter);  
    int right = diameter(root->right, maxDiameter);  
    maxDiameter = max(maxDiameter, left + right);  
    return 1 + max(left, right);  
}
```

```
int diameterOfBinaryTree(TreeNode* root) {  
    int maxDiameter = 0;  
    diameter(root, maxDiameter);  
    return maxDiameter;  
}
```

Dry Run:

Input: [1, 2, 3, 4,

5]

Diameter = max(2, 3).

Output: 3

- Time Complexity: $O(N)$.
- Space Complexity: $O(H)$.

4. Balanced Binary Tree

Approach 1: Recursive Height Check

- For each node, check if the height difference of left and right subtrees is ≤ 1 .

Pseudocode:

```
int height(TreeNode* root) {  
    if (!root) return 0;  
    int left = height(root->left);  
    int right = height(root->right);  
    if (left == -1 or right == -1 or abs(left - right) > 1) return -1;  
    return 1 + max(left, right);  
}
```

```
bool isBalanced(TreeNode* root) {  
    return height(root) != -1;  
}
```

```
-1;  
}
```

Dry Run:

Input: [3, 9, 20, null, null, 15, 7]

Left height = 1, right = 2 → Balanced.

Output: true

- Time Complexity: $O(N)$.

- Space Complexity: $O(H)$.

5. Same Tree

Approach 1: Recursive Comparison

Check if both nodes are identical recursively.

Pseudocode:

```
bool isSameTree(TreeNode* p, TreeNode* q) {  
    if (!p && q) return true;  
    if (!p or q or p->val != q->val) return false;  
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);  
}
```

Dry

Run:

Input: [1, 2, 3], [1, 2, 3]

Compare each node $\rightarrow$ Identical.

Output: true

Time Complexity: $O(N)$.

Space Complexity: $O(H)$.

6. Subtree of Another Tree

Approach 1: Recursive Subtree Match

- Check if the root matches or if the subtree is present in the left/right child.

Pseudocode:

```
bool isSubtree(TreeNode* root, TreeNode* subRoot) {  
    if (!root) return false;  
    if (isSameTree(root, subRoot)) return true;  
    return isSubtree(root->left, subRoot) or isSubtree(root->right, subRoot);  
}
```

Dry Run:

Input: [3, 4, 5, 1, 2], [4, 1, 2]

Match subtree at node

4.

Output: true

- Time Complexity: $O(M \times N)$.

- Space Complexity: $O(H)$.

7. Lowest Common Ancestor of a Binary Search Tree

Approach 1: Recursive

- Traverse based on BST properties.

Pseudocode:

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {  
    if (p->val < root->val && q->val < root->val) {  
        return lowestCommonAncestor(root->left, p, q);  
    }  
    if (p->val > root->val && q->val > root->val) {  
        return lowestCommonAncestor(root->right, p, q);  
    }  
    return root;  
}
```

Dry Run:

Input: [6, 2, 8, 0, 4, 7, 9], p = 2, q =